

University Course Management System

Part 1: Composition & Aggregation

Overview

In this lab you will design and implement a University Course Management System that models how **Students**, **Courses**, **Professors**, and **Departments** relate to one another. The point of the exercise isn't just to get the four classes working — it's to notice that they don't all relate to each other the same way, and to make that difference show up directly in how each class is written.

A Department **owns** its Courses — this is **composition**: a strong, exclusive relationship where the container is responsible for its parts. A Course, on the other hand, merely **refers to** a Professor and some Students — this is **aggregation**: a weak relationship where both sides can exist independently of each other. Both relationships are built entirely out of tools you already know (member variables, pointers, vector/dynamic arrays) — the skill this lab is after is choosing the right one for each relationship, not learning new syntax.

Learning Objectives

- Distinguish composition ("owns-a", strong, lifetime-bound) from aggregation ("has-a", weak, independent lifetimes) and recognize which one a given relationship calls for.
- Implement composition in idiomatic C++ using a member stored by value (e.g. `vector<Course>`), so ownership and automatic cleanup fall out of normal C++ value semantics — no manual `new/delete` required.
- Implement aggregation using raw pointers (e.g. `Professor*`, `vector<Student*>`) that a class stores and uses, but never creates or destroys.
- Practice reading a test driver's calls (constructor arguments, which methods take pointers vs. objects) to reverse-engineer the class interface it expects.
- Draw a UML class diagram that distinguishes composition from aggregation using the standard filled-diamond / open-diamond notation.

Composition vs. Aggregation, at a Glance

	Composition	Aggregation
In this lab	Department ↔ Course	Course ↔ Professor, Course ↔ Student, Department ↔ Professor
Ownership	Strong: the container is responsible for its parts' lifetime	Weak: the two objects' lifetimes are independent
If the container is destroyed...	...its parts are destroyed with it	...the referenced object lives on untouched
Typical C++ shape	A member stored by value, e.g. <code>vector<Course></code>	A member stored as a pointer, e.g. <code>Professor*</code> , <code>vector<Student*></code>

Part 1: The Four Classes

Design and implement Student, Professor, Course, and Department. Keep Student and Professor deliberately simple ("lazy classes") — the interesting design decisions are in Course and Department.

Student

Member	Purpose
Student(string name)	Constructor. Stores the student's name.
string getName() const	Accessor.

Professor

Member	Purpose
Professor(string name)	Constructor. Stores the professor's name.
string getName() const	Accessor.

Course

Member	Purpose
Course(string title, Professor* professor)	Constructor. Stores the title and the pointer it's given — it does not create the Professor, and must never delete it.
Professor* professor (private)	Aggregation. Course only ever points at a Professor that already exists somewhere else (main(), in this lab).
vector<Student*> students (private)	Aggregation. Same idea, one pointer per enrolled student.
void enrollStudent(Student* s)	Appends s to students.
void listStudents() const	Prints the course title, the professor's name, and every enrolled student's name.
string getTitle() const / Professor* getProfessor() const	Accessors — Department's listCourses() needs both.

Department

Member	Purpose
Department(string name)	Constructor. Stores the department's name.
vector<Professor*> professors (private)	Aggregation, exactly like Course's professor pointer — Department references these Professors but does not own them.
vector<Course> courses (private)	Composition. Course objects stored BY VALUE, not by pointer. This one design choice IS the composition relationship: because a vector<Course> owns its elements outright, every Course inside it is automatically destroyed the instant the Department itself is destroyed.
void addProfessor(Professor* p)	Appends the pointer.

<code>void addCourse(const Course& c)</code>	Copies <code>c</code> into <code>courses</code> . (Note: the parameter can be passed by reference for efficiency — that only avoids one extra temporary copy of the argument; <code>push_back</code> into the vector still makes the copy that <code>Department</code> will own.)
<code>void listCourses() const</code>	Prints the department's name, then each course's title and professor's name.

Part 2: Wiring the Relationships Together

Because `courses` is a `vector<Course>` holding real `Course` objects (not pointers), `Department` needs no custom destructor to "clean up" its `courses` — the compiler-generated one already destroys the vector, which destroys every `Course` inside it. Writing an explicit destructor anyway, purely to comment on why there's nothing to do, is good practice here and is what the solution does:

```
~Department() {
    // courses (vector<Course>) destroys its own Course objects
    // automatically - that IS the composition relationship.
    // professors (vector<Professor*>) only discards pointers here;
    // deleting the Professor objects themselves would be a bug,
    // since Department does not own them (aggregation).
}
```

A design question worth thinking through before you code: `main()` calls `csDept.addCourse(course1)` **before** any students are enrolled in `course1`, and it calls `course1.listStudents()` directly on the original `course1` object later — not through the department. Since `addCourse` copies `course1` into the department's own vector, that copy was made *before* enrollment happened, so it will never reflect students enrolled afterward. That is exactly why this lab keeps the two displays separate: `listCourses()` is the department's course *catalog* (title + professor), and `listStudents()` is each course's own *roster*, read from the original object that was actually enrolled into. Keep `listCourses()` to catalog information only, and you'll never run into this.

Part 3: Test Driver

Use the following `main()` to test your solution exactly as given — do not modify it. It creates two `Professors`, a `Department`, two `Courses`, and three `Students`, wires them together, and prints the results.

```
int main()
{
    // Create Professors
    Professor prof1("Dr. Russell");
    Professor prof2("Dr. Ambrosio");

    // Create Department
    Department csDept("Computer Science");
    csDept.addProfessor(&prof1);
    csDept.addProfessor(&prof2);

    // Create Courses
    Course course1("Data Structures", &prof1);
    Course course2("Operating Systems", &prof2);

    // Add Courses to Department
    csDept.addCourse(course1);
    csDept.addCourse(course2);
}
```

```

// Create Students
Student s1("Lisa");
Student s2("Bart");
Student s3("Maggie");

// Enroll Students
course1.enrollStudent(&s1);
course1.enrollStudent(&s2);
course1.enrollStudent(&s3);

course2.enrollStudent(&s2);
course2.enrollStudent(&s3);

// Display Courses and Students
csDept.listCourses();
cout << endl;
course1.listStudents();
cout << endl;
course2.listStudents();

cout << "All done!\n";
}

```

Expected Console Output

```

Department: Computer Science
- Data Structures (Professor: Dr. Russell)
- Operating Systems (Professor: Dr. Ambrosio)

Course: Data Structures (Professor: Dr. Russell)
- Lisa
- Bart
- Maggie

Course: Operating Systems (Professor: Dr. Ambrosio)
- Bart
- Maggie
All done!

```

Exact spacing/wording of your printouts may differ slightly — what matters is that Data Structures lists Lisa, Bart, and Maggie, Operating Systems lists Bart and Maggie, and the department catalog names both courses with their correct professor.

Deliverables

1. A UML class diagram showing all four classes and their relationships, using a filled diamond for the composition edge (Department $\blacklozenge$ — Course) and open diamonds for the three aggregation edges (Course $\diamond$ — Professor, Course $\diamond$ — Student, Department $\diamond$ — Professor).
2. A single .cpp file implementing (using inline class style) Student, Professor, Course, and Department. The .cpp file also includes the provided main() unmodified.
3. Your solution compiles cleanly with no warnings.

Quality Control

- Student and Professor each store a *name* and expose *getName()*.
- Course stores its Professor and its enrolled Students as pointers (aggregation) — it never calls *new* or *delete* on either.
- Department stores its Courses by value in a `vector<Course>` (composition) and its Professors as pointers in a `vector<Professor*>` (aggregation).
- Department's destructor (explicit or compiler-generated) never deletes a Professor pointer.
- `listCourses()` prints the department's course catalog; `listStudents()` prints a specific course's roster, matching the sample output.
- The UML diagram uses the correct diamond (filled vs. open) for each relationship.
- Program output matches the Expected Console Output section above.

Coming in Part 2

This is Part 1 of a multi-part lab. The starter file already notes an extension for later: each Course will eventually require multiple textbooks and an SI tutor (Supplemental Instruction). As you reflect on the relationships you have created, consider whether a Course's connection to its textbooks is more similar to its relationship with a Professor, or to a Department's connection with its Courses. In Part 2, you'll implement the approach you determine is most appropriate.